

COMPLETE GUIDE

Playwright with AI Automation Testing

From Beginner to Job-Ready SDET

LLM Fine-Tuning · Prompt Engineering · RAG · AI Agents
Playwright MCP · TypeScript · GitHub Copilot · CI/CD

Earn up to ■45 LPA with AI Skills

300+ Code Examples · Real-World Use Cases · Step-by-Step Lessons

Table of Contents

PHASE 1 — Programming & Testing Foundations

- JavaScript Basics
- TypeScript Basics
- Async JavaScript
- Node.js Fundamentals
- Software Testing Concepts
- Types of Testing
- Dev Environment Setup

PHASE 2 — Playwright Core

- What is Playwright?
- First Test
- Locators
- Page Actions
- Assertions
- Browser Contexts
- Handling iframes & Popups
- Configuration
- Fixtures & Hooks
- Page Object Model
- API Testing
- Screenshots & Traces

PHASE 3 — Playwright + TypeScript Advanced

- Full TypeScript Setup
- Advanced TypeScript
- Data-Driven Testing
- Network Interception
- Custom Reporters
- Parallel Execution
- Visual Regression

PHASE 4 — AI & LLM Skills

- What are LLMs?
- Prompt Engineering Basics
- Advanced Prompting
- RAG Concepts
- RAG Implementation
- LLM Fine-Tuning
- Fine-Tuning Workflow
- AI Agents
- Building AI Agents
- MCP Protocol
- Playwright MCP

PHASE 5 — GitHub Copilot, CI/CD & DevOps

-
- Git Fundamentals
 - GitHub Copilot
 - Copilot Advanced
 - GitHub Actions
 - Playwright in CI
 - Docker Basics
 - Playwright in Docker
 - SDET Role
 - GenAI in SDET
 - Capstone Project

PHASE 1

Programming & Testing Foundations

Chapter 1

JavaScript Basics

JavaScript is the programming language of the web and the foundation of all modern automation tools including Playwright. Every Playwright test you write will use JavaScript or TypeScript, so a solid understanding of the basics is essential before anything else.

Variables

Variables store data. JavaScript has three ways to declare them:

```
// var - old way, avoid it
var name = "Alice";

// let - use for values that change
let score = 0;
score = 10; // allowed

// const - use for values that do not change
const MAX_RETRIES = 3;
// MAX_RETRIES = 5; // ERROR - cannot reassign
```

Tip: Always prefer const. Use let only when you need to reassign. Never use var in modern code.

Data Types

```
const text = "Hello World"; // String
const count = 42; // Number
const price = 9.99; // Number (decimals too)
const isReady = true; // Boolean
const nothing = null; // Null
let notSet; // Undefined
const user = { name: "Bob", age: 25 }; // Object
const colors = ["red", "green", "blue"]; // Array
```

Functions

```
// Traditional function
function greet(name) {
  return "Hello, " + name;
}

// Arrow function (modern, preferred)
```

Use Case in Playwright

Everything above is used constantly in Playwright tests. Here is a preview of what real test code looks like:

```
// A simple Playwright test (preview)
const urls = ["/login", "/dashboard", "/profile"];

for (const path of urls) {
  await page.goto("https://myapp.com" + path);
  const title = await page.title();
  console.log(path, "->", title);
}
```

Chapter 2

TypeScript Basics

TypeScript is JavaScript with types added. It catches bugs before your code runs and is the standard language for Playwright projects in professional teams.

Why TypeScript?

- Catches type errors at compile time, not runtime
- Autocomplete and IntelliSense in VS Code
- Self-documenting code — types act as documentation
- Industry standard for enterprise test automation

Basic Types

```
// Explicit type annotations
let username: string = "Alice";
let age: number = 30;
let isActive: boolean = true;

// Arrays
let scores: number[] = [90, 85, 78];
let names: string[] = ["Bob", "Carol"];

// Function with typed params + return
function add(a: number, b: number): number {
  return a + b;
}

// TypeScript infers types when obvious
let city = "Chennai"; // inferred: string
let count = 42; // inferred: number
```

Interfaces — Defining Object Shapes

```
interface User {
  id: number;
  name: string;
  email: string;
  role?: string; // ? means optional
}

const user: User = {
  id: 1,
  name: "Alice",
  email: "alice@test.com",
```

```
};

// Use in Playwright test data
interface LoginCredentials {
  username: string;
  password: string;
}

async function login(page: Page, creds: LoginCredentials) {
  await page.fill("#username", creds.username);
  await page.fill("#password", creds.password);
  await page.click("#submit");
}
```

Generics

```
// Generic function works with any type
function getFirst<T>(arr: T[]): T {
  return arr[0];
}

getFirst<string>(["a", "b", "c"]); // "a"
getFirst<number>([1, 2, 3]); // 1

// Common in Playwright fixtures
async function getData<T>(url: string): Promise<T> {
  const response = await fetch(url);
  return response.json() as T;
}
```

Chapter 3

Async JavaScript

Every single Playwright action is asynchronous. Understanding `async/await` is not optional — it is the most critical concept for writing automation tests.

The Problem: Why Async?

Browsers take time to load pages, click buttons, and fetch data. If JavaScript waited for each operation, everything would freeze. Async code lets your program continue while waiting.

Promises

```
// A Promise represents a future value
const promise = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Data loaded!"), 2000);
});

// .then() handles success
promise.then(data => console.log(data));
// After 2 seconds: "Data loaded!"
```

`async / await` — The Modern Way

```
// async makes a function return a Promise
// await pauses until Promise resolves

async function fetchUser(id: number) {
  const response = await fetch(`/api/users/${id}`);
  const data = await response.json();
  return data;
}

// Usage
const user = await fetchUser(1);
console.log(user.name);
```

Error Handling with `try/catch`

```
async function loginTest(page: Page) {
  try {
    await page.goto("https://myapp.com/login");
    await page.fill("#email", "user@test.com");
    await page.click("#submit");
    await page.waitForURL("*/dashboard");
    console.log("Login successful");
  } catch (error) {
```

```
console.error("Login failed:", error);
throw error; // re-throw so test fails
}
}
```

Parallel Execution with Promise.all

```
// Run multiple async tasks AT THE SAME TIME
const [user, products, orders] = await Promise.all([
  fetchUser(1),
  fetchProducts(),
  fetchOrders(1),
]);

// In Playwright - navigate and wait simultaneously
await Promise.all([
  page.waitForNavigation(),
  page.click("#submit"),
]);
```

Chapter 4

Node.js Fundamentals

Node.js lets JavaScript run outside the browser — on your computer. Playwright runs on Node.js. Understanding Node.js basics helps you manage projects, read files, and set up environments.

npm — Node Package Manager

```
# Initialize a new project
npm init -y

# Install Playwright
npm install @playwright/test

# Install a package
npm install dotenv

# Install dev-only package
npm install --save-dev typescript

# Run a script defined in package.json
npm test
npm run test:headed
```

package.json — Project Config

```
{
  "name": "my-playwright-project",
  "scripts": {
    "test": "playwright test",
    "test:headed": "playwright test --headed",
    "test:ui": "playwright test --ui",
    "report": "playwright show-report"
  },
  "dependencies": {
    "@playwright/test": "^1.44.0"
  }
}
```

Reading Files & Environment Variables

```
// Read a JSON test data file
import fs from "fs";

const rawData = fs.readFileSync("./data/users.json", "utf-8");
const users = JSON.parse(rawData);

// Environment variables (store secrets safely)
```


Chapter 5

Software Testing Concepts

Before writing a single line of automation code, you must understand WHY we test software and what problem automation solves.

Why Test?

- Catch bugs before users do
- Ensure new features do not break existing ones
- Build confidence to deploy faster
- Document expected application behavior

Testing Pyramid

```
/\
/E2E\ <- Few, slow, expensive
/-----\
/Integration\ <- Some, medium speed
/-----\
/ Unit Tests \ <- Many, fast, cheap
/_____ \
```

Unit tests test one function. Integration tests test how pieces work together. E2E (End-to-End) tests simulate a real user — this is where Playwright lives.

Use Case: When to Automate

Scenario	Automate?	Why
Login with valid credentials	Yes	Run on every deploy
Exploratory testing new UI	No	Human judgment needed
Regression test of 50 pages	Yes	Too slow manually
One-off bug verification	No	Not worth the time
Cross-browser testing	Yes	Playwright excels here

PHASE 2

Playwright Core

Chapter 6

What is Playwright?

Playwright is an open-source automation library created by Microsoft. It lets you control real browsers (Chrome, Firefox, Safari) programmatically — simulating exactly what a real user does.

Playwright vs Other Tools

Feature	Playwright	Selenium	Cypress
Browsers	Chrome, Firefox, Safari	Chrome, Firefox, Safari, IE	Chrome, Firefox, Edge
Speed	Very Fast	Slow	Fast
Auto-wait	Built-in	Manual waits needed	Built-in
API testing	Yes	No	Limited
Parallel	Native	Grid needed	Limited
Language	JS/TS/Python/Java/C#	Many	JS/TS only
MCP/AI	Yes (2024)	No	No

Installation

```
# Step 1: Create project folder
mkdir my-tests && cd my-tests

# Step 2: Initialize npm project
npm init -y

# Step 3: Install Playwright
npm init playwright@latest
# Choose: TypeScript, tests/ folder, GitHub Actions: No

# Step 4: Install browsers
npx playwright install

# Step 5: Run example tests
npx playwright test
```

Chapter 7

Writing Your First Test

A Playwright test has three parts: arrange (set up), act (do things), and assert (verify). Let us write a real test from scratch.

Your First Test File

```
// tests/first.spec.ts
import { test, expect } from "@playwright/test";

test("homepage has correct title", async ({ page }) => {
  // ARRANGE: navigate to the page
  await page.goto("https://playwright.dev");

  // ACT: (nothing to do for this simple test)

  // ASSERT: check the page title
  await expect(page).toHaveTitle(/Playwright/);
});

test("get started link works", async ({ page }) => {
  await page.goto("https://playwright.dev");

  // Find and click the link
  await page.getByRole("link", { name: "Get started" }).click();

  // Verify we landed on the right page
  await expect(page).toHaveURL(/intro/);
});
```

Running Tests

```
# Run all tests
npx playwright test

# Run with browser visible
npx playwright test --headed

# Run a specific file
npx playwright test tests/first.spec.ts

# Run in interactive UI mode
npx playwright test --ui

# Show HTML report after run
npx playwright show-report
```

Chapter 8

Locators — Finding Elements

A locator tells Playwright which element on the page to interact with. Choosing the right locator is one of the most important skills in automation.

Recommended Locators (use these first)

```
// By role - best practice, matches accessibility tree
page.getByRole("button", { name: "Submit" })
page.getByRole("textbox", { name: "Email" })
page.getByRole("heading", { name: "Dashboard" })

// By label - great for form inputs
page.getByLabel("Username")
page.getByLabel("Password")

// By placeholder text
page.getByPlaceholder("Enter your email")

// By text content
page.getByText("Welcome back!")

// By test ID - most stable, set by developers
page.getByTestId("login-button")
// HTML: <button data-testid="login-button">
```

Fallback Locators

```
// CSS selectors
page.locator("#submit-btn") // by ID
page.locator(".btn-primary") // by class
page.locator("input[type=email]") // by attribute

// XPath (use as last resort)
page.locator("//button[text()='Submit']")

// Chaining locators
page.locator(".login-form").getByRole("button")

// nth element
page.locator("li").nth(2) // 3rd item (0-indexed)
page.locator("li").first()
page.locator("li").last()
```

Use Case: Login Form

Chapter 9

Page Actions

Actions are things Playwright does on the page — clicking, typing, hovering, selecting. Playwright automatically waits for elements to be ready before acting.

```
// Click
await page.getByRole("button", { name: "Save" }).click();
await page.getByText("Terms").dblclick(); // double click
await page.getByText("Item").click({ button: "right" }); // right click

// Type / Fill
await page.getByLabel("Name").fill("Alice"); // clears then fills
await page.getByLabel("Name").type("Alice"); // types char by char
await page.getByLabel("Name").pressSequentially("Alice"); // simulates keyboard

// Clear input
await page.getByLabel("Name").clear();

// Select dropdown
await page.getByLabel("Country").selectOption("India");
await page.getByLabel("Size").selectOption({ label: "Large" });

// Checkbox & Radio
await page.getByLabel("Remember me").check();
await page.getByLabel("Remember me").unchecked();
await expect(page.getByLabel("Terms")).toBeChecked();

// Hover
await page.getByText("Menu").hover();

// Keyboard
await page.keyboard.press("Enter");
await page.keyboard.press("Tab");
await page.keyboard.down("Shift");

// File upload
await page.getByLabel("Upload").setInputFiles("./test-file.pdf");

// Navigation
await page.goto("https://example.com");
await page.goBack();
await page.goForward();
await page.reload();
```

Chapter 10

Assertions

Assertions verify that the application behaves correctly. Playwright has smart auto-retrying assertions — they keep checking until the condition is true or timeout expires.

```
// Page-level assertions
await expect(page).toHaveTitle("My App");
await expect(page).toHaveURL("https://app.com/dashboard");

// Element visibility
await expect(page.getByText("Welcome")).toBeVisible();
await expect(page.getByText("Error")).toBeHidden();
await expect(page.getByRole("button")).toBeEnabled();
await expect(page.getByRole("button")).toBeDisabled();

// Text content
await expect(page.getByTestId("total")).toHaveText("$49.99");
await expect(page.getByTestId("msg")).toContainText("success");

// Input values
await expect(page.getByLabel("Email")).toHaveValue("alice@test.com");

// Count elements
await expect(page.locator("li")).toHaveCount(5);

// Attribute
await expect(page.getByRole("img")).toHaveAttribute("alt", "Logo");

// Negative assertions
await expect(page.getByText("Error")).not.toBeVisible();

// Soft assertions (collect all failures)
await expect.soft(page.getByText("Title")).toBeVisible();
await expect.soft(page.getByText("Price")).toBeVisible();
// Test continues even if first assertion fails
```

Chapter 11

Page Object Model (POM)

The Page Object Model is a design pattern that organizes your tests. Instead of putting locators directly in tests, you create a class for each page. This makes tests readable, maintainable, and reusable.

Without POM — Messy

```
// BAD - locators scattered everywhere
test("login", async ({ page }) => {
  await page.goto("/login");
  await page.locator("#email").fill("user@test.com");
  await page.locator("#pass").fill("secret");
  await page.locator(".btn-login").click();
  await expect(page.locator(".dashboard-title")).toBeVisible();
});
```

With POM — Clean & Maintainable

```
// pages/LoginPage.ts
import { Page, Locator } from "@playwright/test";

export class LoginPage {
  readonly page: Page;
  readonly email: Locator;
  readonly password: Locator;
  readonly submit: Locator;

  constructor(page: Page) {
    this.page = page;
    this.email = page.getByLabel("Email");
    this.password = page.getByLabel("Password");
    this.submit = page.getByRole("button", { name: "Sign in" });
  }

  async goto() { await this.page.goto("/login"); }

  async login(email: string, password: string) {
    await this.email.fill(email);
    await this.password.fill(password);
    await this.submit.click();
  }
}

// tests/login.spec.ts
import { test, expect } from "@playwright/test";
import { LoginPage } from "../pages/LoginPage";
```

PHASE 3

Playwright + TypeScript Advanced

Chapter 12

Data-Driven Testing

Run the same test logic with multiple sets of data. This dramatically increases coverage without duplicating code.

```
// tests/login.spec.ts
import { test, expect } from "@playwright/test";

const testCases = [
  { email: "admin@test.com", pass: "Admin123", role: "admin", expectURL: "/admin" },
  { email: "user@test.com", pass: "User123", role: "user", expectURL: "/home" },
  { email: "guest@test.com", pass: "Guest123", role: "guest", expectURL: "/guest" },
];

for (const tc of testCases) {
  test(`${tc.role} can login and lands on ${tc.expectURL}`, async ({ page }) => {
    await page.goto("/login");
    await page.getByLabel("Email").fill(tc.email);
    await page.getByLabel("Password").fill(tc.pass);
    await page.getByRole("button", { name: "Sign in" }).click();
    await expect(page).toHaveURL(new RegExp(tc.expectURL));
  });
}
```

Loading Data from JSON File

```
// test-data/products.json
// [{ "name": "Laptop", "price": "999", "sku": "LAP001" }, ...]

import testData from "../test-data/products.json";

for (const product of testData) {
  test(`product ${product.sku} displays correctly`, async ({ page }) => {
    await page.goto(`/products/${product.sku}`);
    await expect(page.getByTestId("product-name")).toHaveText(product.name);
    await expect(page.getByTestId("price")).toContainText(product.price);
  });
}
```

Chapter 13

Network Interception

Playwright can intercept, mock, and modify network requests. This lets you test UI with controlled API responses — even error scenarios.

```
// Mock an API response
test("shows products from API", async ({ page }) => {
  // Intercept before navigating
  await page.route("**/api/products", async route => {
    await route.fulfill({
      status: 200,
      contentType: "application/json",
      body: JSON.stringify([
        { id: 1, name: "Test Product", price: 99 }
      ]),
    });
  });

  await page.goto("/products");
  await expect(page.getByText("Test Product")).toBeVisible();
});

// Simulate API error
test("shows error when API fails", async ({ page }) => {
  await page.route("**/api/products", route =>
    route.fulfill({ status: 500, body: "Server Error" })
  );

  await page.goto("/products");
  await expect(page.getByText("Something went wrong")).toBeVisible();
});

// Block all image requests (speed up tests)
await page.route("**/*.{png,jpg,jpeg,gif,svg}", r => r.abort());
```

PHASE 4

AI & LLM Skills

Chapter 14

What are LLMs?

Large Language Models (LLMs) are AI systems trained on massive amounts of text. They predict the next token (word/subword) in a sequence — which turns out to be enough to answer questions, write code, and reason through problems.

How LLMs Work (Simply)

- Training: Model reads billions of web pages, books, and code
- Learns patterns: Predicts what word comes next with high accuracy
- At inference: You give it a prompt, it generates a response token by token
- Context window: How much text it can "see" at once (e.g. 200,000 tokens)

Popular LLMs

Model	Company	Best For
GPT-4o	OpenAI	General tasks, coding, vision
Claude 3.5	Anthropic	Long docs, safety, reasoning
Gemini 1.5	Google	Multimodal, large context
Llama 3	Meta	Open source, self-hosted
Mistral	Mistral AI	Fast, efficient, open source

Chapter 15

Prompt Engineering

Prompt engineering is the skill of writing instructions for LLMs to get the best output. It is one of the highest-leverage skills in AI — small changes in wording can dramatically change the result.

Zero-Shot Prompting

```
// Just ask directly – no examples given
Prompt: "Write a Playwright test for a login page"

// Better zero-shot with context
Prompt: "You are an expert QA engineer.
Write a Playwright TypeScript test that:
- Goes to /login
- Fills email with user@test.com
- Fills password
- Clicks Submit
- Asserts URL changes to /dashboard
Use getByRole and getByLabel locators only."
```

Few-Shot Prompting

```
// Give examples to establish pattern
Prompt:
"Convert these test descriptions to Playwright code.
Follow the examples exactly.

Example 1:
Description: Click the Save button
Code: await page.getByRole("button", { name: "Save" }).click();

Example 2:
Description: Fill the Name field with Alice
Code: await page.getByLabel("Name").fill("Alice");

Now convert:
Description: Check that the success message is visible"
```

Chain-of-Thought Prompting

```
// Ask the model to reason step by step
Prompt:
"I have a test that is flaky – it fails randomly.
Think through the possible causes step by step,
then suggest how to fix each one.

The test clicks a button and then checks
```

```
if a modal appears. It passes 8 out of 10 times."
```

```
// Model will reason:  
// Step 1: The element might not be ready...  
// Step 2: Race condition between click and modal...  
// Step 3: Solution: wait for modal with expect()...
```

Role Prompting

```
// Assign a role for better responses  
System: "You are a Senior SDET with 10 years of  
Playwright experience. You write clean, maintainable  
tests following the Page Object Model pattern.  
Always add meaningful comments."  
  
User: "Write tests for an e-commerce checkout flow"
```

Chapter 16

RAG — Retrieval Augmented Generation

RAG is a technique where you give an LLM access to a private knowledge base (your documents, database, code) so it can answer questions about data it was never trained on.

The RAG Architecture

```
User Question
|
v
[1. Embed question into vector]
|
v
[2. Search vector DB for similar chunks]
|
v
[3. Retrieve top-K relevant chunks]
|
v
[4. Inject chunks into LLM prompt]
|
v
[5. LLM generates answer using context]
|
v
Answer (grounded in your documents)
```

RAG Use Cases in Testing

- AI bot that answers "How do I test feature X?" using your test docs
- Auto-generate tests by querying your API specification documents
- Bug analysis bot that searches historical bug reports
- Test coverage assistant that knows your entire codebase

Simple RAG Implementation

```
// Conceptual RAG pipeline in TypeScript
import OpenAI from "openai";

// Step 1: Store your documents as embeddings
async function embedDocument(text: string) {
  const res = await openai.embeddings.create({
    model: "text-embedding-3-small",
    input: text,
  });
}
```

Chapter 17

AI Agents

An AI agent is an LLM that can take actions — not just answer questions. It uses tools (functions), makes decisions, and loops until a task is complete. This is the frontier of AI in 2024–2025.

Agent Components

- Brain: The LLM (GPT-4, Claude, etc.)
- Tools: Functions the agent can call (search, code execution, browser)
- Memory: Short-term (conversation) and long-term (vector DB)
- Planning: Breaking tasks into subtasks (ReAct, Chain-of-Thought)

ReAct Pattern — Reason + Act

```
// The agent loops: Thought -> Action -> Observation

Thought: I need to test the login page.
First I should navigate to it.

Action: browser.goto("https://app.com/login")

Observation: Page loaded. Has email and password fields.

Thought: Now I should fill in the credentials.

Action: browser.fill("#email", "user@test.com")

Observation: Email field filled.

... continues until task complete
```

Building a Simple Agent

```
const tools = [
  {
    name: "run_playwright_test",
    description: "Run a Playwright test and return results",
    parameters: { testCode: "string" }
  },
  {
    name: "read_file",
    description: "Read a file from the project",
    parameters: { path: "string" }
  }
];
```

Chapter 18

MCP & Playwright MCP

Model Context Protocol (MCP) is an open standard by Anthropic that lets AI models connect to tools and data sources in a standardized way. Playwright MCP lets an AI agent control a real browser.

What is MCP?

Before MCP, every AI tool integration was custom-built. MCP standardizes how AI models talk to external tools — like a USB standard, but for AI.

Playwright MCP — AI Controls the Browser

```
# Install Playwright MCP server
npm install @playwright/mcp

// Configure in your AI agent (e.g. Claude Desktop)
// mcp_config.json
{
  "servers": {
    "playwright": {
      "command": "npx",
      "args": ["@playwright/mcp"]
    }
  }
}

// Now you can tell Claude:
// "Go to amazon.in and find the price of iPhone 15"
// Claude will literally control a browser and do it
```

Use Cases for Playwright MCP

- AI agent that automatically tests your website after each deploy
- Bug reproduction: describe a bug, agent reproduces it
- Auto-generate test scripts by watching agent navigate your app
- AI-powered web scraping with natural language instructions

PHASE 5

GitHub Copilot, CI/CD & DevOps

Chapter 19

Git Fundamentals

Git is version control — it tracks every change to your code. It is essential for every software job. GitHub hosts your code online and enables team collaboration.

```
# One-time setup
git config --global user.name "Your Name"
git config --global user.email "you@email.com"

# Start a project
git init # initialize repo
git clone <url> # copy existing repo

# Daily workflow
git status # see what changed
git add . # stage all changes
git add tests/login.spec.ts # stage one file
git commit -m "add login test" # save snapshot
git push origin main # upload to GitHub
git pull # download latest changes

# Branching (work without affecting main)
git checkout -b feature/login-tests # create & switch
git checkout main # go back to main
git merge feature/login-tests # merge your work

# See history
git log --oneline
```

Chapter 20

GitHub Copilot

GitHub Copilot is an AI pair programmer. It suggests code as you type, generates entire functions from comments, and can write Playwright tests from plain English descriptions.

Setting Up Copilot

- Install "GitHub Copilot" extension in VS Code
- Sign in with GitHub account (requires subscription)
- Enable Copilot Chat for conversational AI in VS Code

Copilot Writing Tests For You

```
// Just type this comment and press Tab...
// Test that the shopping cart correctly calculates
// total price including 18% GST

// Copilot suggests:
test("cart calculates total with GST", async ({ page }) => {
  await page.goto("/shop");
  await page.getByText("Add to Cart").first().click();
  await page.goto("/cart");
  const subtotal = await page.getByTestId("subtotal").textContent();
  const gst = parseFloat(subtotal!) * 0.18;
  const expected = (parseFloat(subtotal!) + gst).toFixed(2);
  await expect(page.getByTestId("total")).toContainText(expected);
});
```

Copilot Chat — Natural Language to Code

```
// In VS Code Copilot Chat, type:
"Generate a Playwright test for user registration
that tests: valid signup, duplicate email error,
and password mismatch validation"

// Copilot generates all three test cases
// with proper assertions and error messages
```

Chapter 21

GitHub Actions & CI/CD

CI/CD (Continuous Integration / Continuous Deployment) automatically runs your tests every time code changes. GitHub Actions is the most popular CI/CD tool and is free for public repos.

What is CI/CD?

Every time a developer pushes code, GitHub Actions automatically: runs all tests, reports pass/fail, and blocks the merge if tests fail. This catches bugs before they reach production.

Playwright GitHub Actions Workflow

```
# .github/workflows/playwright.yml
name: Playwright Tests

on:
  push:
    branches: [main]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20

      - name: Install dependencies
        run: npm ci

      - name: Install Playwright browsers
        run: npx playwright install --with-deps

      - name: Run Playwright tests
        run: npx playwright test

      - name: Upload test report
        uses: actions/upload-artifact@v4
        if: always()
        with:
          name: playwright-report
```

Chapter 22

Docker & Playwright in Docker

Docker packages your app and its dependencies into a container — a self-contained box that runs identically everywhere. This solves "it works on my machine" problems in CI/CD.

Key Concepts

- Image: Blueprint for a container (like a class)
- Container: Running instance of an image (like an object)
- Dockerfile: Instructions to build an image
- docker-compose: Run multiple containers together

Playwright Dockerfile

```
# Dockerfile
FROM mcr.microsoft.com/playwright:v1.44.0-jammy

WORKDIR /app

# Copy package files
COPY package*.json ./

# Install dependencies
RUN npm ci

# Copy test files
COPY . .

# Run tests
CMD ["npx", "playwright", "test"]

# Build and run
# docker build -t my-tests .
# docker run my-tests
```

Chapter 23

SDET Role & GenAI in Testing

An SDET (Software Development Engineer in Test) writes both production code and test code. With AI skills added, you become one of the most in-demand profiles in the industry.

SDET vs Manual Tester vs Automation Engineer

Skill	Manual Tester	Automation QA	SDET
Exploratory testing	Yes	Some	Yes
Write automation code	No	Yes	Yes
Write production code	No	No	Yes
CI/CD pipelines	No	Basic	Yes
Test framework design	No	Some	Yes
LLM / AI skills	No	Rare	Yes (2025)
Salary (India)	3-6 LPA	6-12 LPA	15-45 LPA

GenAI Superpowers for SDETs

- Auto-generate test cases from user stories using LLMs
- Self-healing locators — AI finds new selector when old one breaks
- Natural language test creation — "test the checkout flow"
- AI-powered visual testing — detect UI regressions automatically
- Generate test data intelligently using AI

Self-Healing Locator Example

```
// When a locator breaks, AI suggests the fix
// Old: page.locator("#submit-btn") <- broken, ID changed

// AI analyzes the page DOM and suggests:
// page.getByRole("button", { name: "Submit" })
// Because: role-based locators are more stable

// Auto-heal with Copilot (comment-driven)
// TODO: this locator is broken, fix it
// page.locator("#old-broken-id").click()

// Copilot suggests based on page context:
await page.getByTestId("submit").click();
```

Chapter 24

Capstone Project

Build a complete, production-grade test automation framework from scratch. This project covers everything you have learned and is what you show in interviews.

Project: E-Commerce Test Suite with AI

```
project/
├── pages/
│   ├── LoginPage.ts
│   ├── ProductPage.ts
│   ├── CartPage.ts
│   └── CheckoutPage.ts
├── tests/
│   ├── auth/login.spec.ts
│   ├── products/search.spec.ts
│   ├── cart/cart.spec.ts
│   ├── checkout/payment.spec.ts
│   ├── fixtures/custom-fixtures.ts
│   ├── test-data/users.json
│   ├── utils/api-helper.ts
│   ├── .github/workflows/playwright.yml
│   ├── playwright.config.ts
│   └── Dockerfile
```

playwright.config.ts — Full Setup

```
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests",
  timeout: 30_000,
  retries: process.env.CI ? 2 : 0,
  workers: process.env.CI ? 4 : 2,
  reporter: [
    ["html"],
    ["list"],
  ],

  use: {
    baseURL: process.env.BASE_URL || "http://localhost:3000",
    trace: "on-first-retry",
    screenshot: "only-on-failure",
    video: "retain-on-failure",
  },

  projects: [
    { name: "chromium", use: { ...devices["Desktop Chrome"] } },
    { name: "firefox", use: { ...devices["Desktop Firefox"] } },
    { name: "webkit", use: { ...devices["Desktop Safari"] } },
  ],
});
```

```
{ name: "mobile", use: { ...devices["iPhone 13"] } },  
],  
});
```

What to Include in Your Portfolio

- 10+ test specs covering real user journeys
- Page Object Model with TypeScript interfaces
- API tests alongside UI tests
- GitHub Actions workflow running on every push
- Allure or HTML test report published to GitHub Pages
- README with architecture diagram and run instructions

Your Learning Path Forward

You now have a complete roadmap from beginner JavaScript to AI-powered test automation. Here is how to turn this knowledge into a job:

30-Day Quick Start Plan

Week	Focus	Deliverable
Week 1	JS/TS + Playwright basics	5 working test files
Week 2	POM + API testing	Mini framework for 1 app
Week 3	CI/CD + GitHub Actions	Auto-running pipeline
Week 4	Prompt engineering + AI agents	AI-assisted test suite

Good luck on your journey to ■45 LPA!
Playwright + AI Skills = High-Value SDET